Available online at www.sciencedirect.com
SCIENCE DIRECT®
Pattern Recognition Letters 27 (2006) 1266–1274
Pattern Recognition Letters
www.elsevier.com/locate/patrec

Using Gaussian distribution to construct fitness functions in genetic programming for multiclass object classification

Mengjie Zhang *, Will Smart

School of Mathematics, Statistics and Computer Science, Victoria University of Wellington, P.O. Box 600, Wellington, New Zealand

Available online 4 January 2006

Abstract

This paper describes a new approach to the use of Gaussian distribution in genetic programming (GP) for multiclass object classification problems. Instead of using predefined multiple thresholds to form different regions in the program output space for different classes, this approach uses probabilities of different classes, derived from Gaussian distributions, to construct the fitness function for classification. Two fitness measures, overlap area and weighted distribution distance, have been developed. Rather than using the best evolved program in a population, this approach uses multiple programs and a voting strategy to perform classification. The approach is examined on three multiclass object classification problems of increasing difficulty and compared with a basic GP approach. The results suggest that the new approach is more effective and more efficient than the basic GP approach. Although developed for object classification, this approach is expected to be able to be applied to other classification problems.

Keywords: Probability based genetic programming; Object recognition; Object detection; Fitness function; Multiclass classification

1. Introduction

Classification tasks arise in a very wide range of applications, such as detecting faces from video images, recognising words in streams of speech, diagnosing medical conditions from the output of medical tests, and detecting fraudulent credit card transactions (Eggermont et al., 1999; Gray et al., 1996; Valentin et al., 1994). In many cases, people (possibly highly trained experts) are able to perform the classification task well, but there is either a shortage of such experts, or the cost of people is too high. Given the amount of data that needs to be classified, automated classification systems are highly desirable. However, creating automated classification systems that have sufficient accuracy and reliability turns out to be very difficult.

Genetic programming (GP) is a relatively recent and fast developing approach to automatic programming (Banzhaf et al., 1998; Koza, 1992, 1994). In GP, solutions to a problem can be represented in different forms but are usually interpreted as computer programs. Darwinian principles of natural selection and recombination are used to evolve a population of programs towards an effective solution to specific problems. The flexibility and expressiveness of computer program representation, combined with the powerful capabilities of evolutionary search, make GP an exciting new method to solve a great variety of problems.

GP research has considered a variety of kinds of classifier programs, using different program representations, including decision tree classifiers, classification rule sets (Koza, 1994), and linear and graph classifiers (Banzhaf et al., 1998). Recently, a new form of classifier representation—numeric expression (tree-like) classifiers—has been developed using GP (Loveard et al., 2001; Song et al., 2002; Tackett, 1993; Zhang and Ciesielski, 1999). This form has been successfully applied to real world classification problems such as detecting and recognising particular classes of objects in images (Song et al., 2002; Tackett, 1993; Zhang et al., 2003a,b), demonstrating the potential of GP as a general method to solve classification problems.

* Corresponding author.
E-mail address: mengjie@mcs.vuw.ac.nz (M. Zhang).

0167-8655/$ - see front matter © 2005 Elsevier B.V. All rights reserved.
doi:10.1016/j.patrec.2005.07.024

The output of a numeric expression GP classifier is a numeric value that is typically translated into a class label. For the simple binary classification case, this translation can be based on the sign of the numeric value (Song et al., 2002; Loveard et al., 2001; Tackett, 1993; Howard et al., 1999; Sherrah et al., 1997; Smart and Zhang, 2003; Zhang and Smart, 2004); for multiclass problems, finding the appropriate boundary values to separate the different classes is more difficult. The emphasis of previous approaches was often on separating the program output space into regions (referred to as the *basic GP approach*), with each region indicating a different class. This includes a primary static method such as object classification map or static range selection (Zhang and Ciesielski, 1999; Loveard et al., 2001; Zhang et al., 2003b), dynamic range selection (Loveard et al., 2001), centred and slotted dynamic class boundary determination methods (Smart and Zhang, 2003; Zhang and Smart, 2004). Past work has demonstrated the effectiveness of these approaches, particularly the dynamic methods, on a number of object classification problems.

In the static methods, the region boundaries of program output space were fixed and predefined. In the dynamic methods, class boundaries were automatically found during the evolutionary process. While the static methods often need a hand-crafting of good boundaries, the dynamic methods usually involve a long time search to automatically find good boundaries. Both approaches usually take very long training times and often result in unnecessarily complex programs, and sometimes poor performances (Zhang et al., 2003b; Zhang and Smart, 2004).

1.1. Goals

To avoid these problems and to improve the classification accuracy and the efficiency of the GP system, this paper aims to investigate a new approach to the use of Gaussian distribution and probability for constructing the class translation rule and the fitness function in genetic programming for multiclass classification problems. Rather than setting up class region boundaries for classification, this approach uses Gaussian distribution to model the behaviour of each program based on the training examples for each class. Instead of using the single best evolved program in a population, this approach uses multiple evolved programs to perform classification. This approach is examined on three multiclass object classification tasks of increasing difficulty and compared with the basic GP approach (Zhang et al., 2003b) on the same tasks. Specifically, we are interested in assessing the following questions:

- How can the fitness function be constructed using the Gaussian distribution?
- How can the classification accuracy be calculated based on the Gaussian distribution and multiple evolved programs?
- Can this approach do a good enough job on the given problems?
- Will the new approach outperform the basic GP approach for the same problems?

Note that this paper is focused on numeric expression (tree-like) genetic programming. While other representations of GP such as linear GP and graph GP even other methods such as Naive Bayes, decision tress and neural networks can also be used for classification problems, such an investigation is beyond the scope of this paper. However, to give an overview of these methods for related tasks, the next subsection briefly reviews the object recognition and classification tasks, with a little summary of different techniques that have been classically employed.

1.2. Related work

Object recognition, or called automatic object recognition or automatic target recognition, is a specific field and a challenging problem in computer vision and image understanding (Forsyth and Ponce, 2003). This task often involves *object localisation* and *object classification*. Object localisation refers to the task of identifying the positions of the objects of interest in a sequence of images either within the visual or infrared spectral bands. Object classification refers to the task of discriminating between images of different kinds of objects, where each image contains only one of the objects of interest.

Traditionally, most researches on object recognition involves four stages: *preprocessing*, *segmentation*, *feature extraction* and *classification* (Caelli and Bischof, 1997; Gose et al., 1996). The preprocessing stage aims to remove noise or enhance edges. In the segmentation stage, a number of coherent regions and "suspicious" regions which might contain objects are usually located and separated from the entire images. The feature extraction stage extracts domain specific features from the segmented regions. Finally, the classification stage uses these features to distinguish the classes of the objects of interest. The features extracted from the images and objects are generally domain specific such as high level relational image features. Data mining and machine learning algorithms are usually applied to object classification.

Object recognition has been of tremendous importance in many application domains. These domains include military applications (Howard et al., 1998; Wong and Sundareshan, 1998; Tackett, 1993), human face recognition (Valentin et al., 1994; Teller and Veloso, 1995), agricultural product classification (Winter et al., 1996), handwritten character recognition (Andre, 1994; LeCun et al., 2001), medical image analysis (Verma, 1998), postal code recognition (LeCun et al., 1989; Ridder et al., 1996), and texture classification (Song et al., 2001).

Since the 1990s, many methods have been employed for object recognition. These include different kind of neural networks (Azimi-Sadjadi et al., 2000; Ridder et al., 1996;

Stahl et al., 2000; Wessels and Omlin, 2000), genetic algorithms (Bala et al., 1997; Huang and liu, 1997), decision trees (Russell and Norvig, 2003), statistical methods such as Gaussian models and Naive Bayes (Dunham, 2003; Russell and Norvig, 2003), support vector machines (Dunham, 2003; Russell and Norvig, 2003), genetic programming (Howard et al., 1999, 2002; Teller and Veloso, 1995; Winkeler and Manjunath, 1997), and hybrid methods (Korning, 1995; Ciesielski and Riley, 1998; Yao and Liu, 1997).

Notice that Gaussian member functions have also been used in fuzzy logic and uncertainty. In this paper, we employ the Gaussian model to genetic programming for designing and constructing the fitness function of a genetic program for multiclass object classification.

1.3. Structure

The rest of the paper is organised as follows. Section 2 describes the basic GP approach to classification. Section 3 describes the new fitness function for classification. Section 4 describes how to calculate the classification accuracy using multiple genetic programs. Section 5 describes the object classification problems to be applied. Section 6 presents the results and Section 7 gives the conclusions.

2. GP applied to multiclass classification—the basic GP approach

In the basic approach, we used the numeric expression (tree-like) genetic programs (Koza, 1992). The ramped half-and-half method was used for generating programs in the initial population and for the mutation operator (Banzhaf et al., 1998). The proportional selection mechanism and the reproduction, crossover and mutation operators (Koza, 1994) were used in the evolutionary learning process.

2.1. Terminals and functions

2.1.1. Terminals

Terminals correspond to image features and form the inputs from the environment. To avoid the problem of hand-crafting of feature extraction programs for obtaining high level, domain specific image features for object classification, the basic GP approach used pixel level, domain independent statistical features as terminals and we expect the GP evolutionary process can automatically select features that are relevant to a particular domain to construct good genetic programs.

The terminals considered in this approach are the means and variances of certain regions in object cutout images. Two such regions were used, the entire object cutout image and the central square region, as shown in Fig. 1. This makes four feature terminals. The values of the feature terminals would remain unchanged in the evolutionary process, but different objects usually have different feature values.

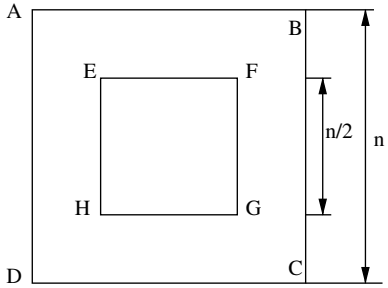

Fig. 1. Region features as terminals.

Notice that these features might not be sufficient for some difficult object classification problems. However, they have been found reasonable in many problems and the selection of good features is not the goal of this paper.

In addition, we also used some constants as terminals. These constants are randomly generated using a uniform distribution at the beginning of evolution. Unlike feature terminals, the values of this kind of terminals are the same for all object images.

2.1.2. Functions

In the function set, the four standard arithmetic and a conditional operation were used to form the function set:

$$\text{FuncSet} = \{+, -, *, /, if\} \quad (1)$$

the $+$, $-$, $/$ and $*$ operators are addition, subtraction, multiplication and "protected" division with two arguments. The "protected" division is the usual division operator except that a divide by zero gives a result of zero. The *if* function takes three arguments. If the first argument is negative, the *if* function returns its second argument; otherwise, it returns its third argument. The *if* function allows a program to contain a different expression in different regions of the feature space, and allows discontinuous programs, rather than insisting on smooth functions. All functions can take the result of any other functions or terminals as arguments.

2.2. Fitness function

In the basic GP approach, we used classification accuracy on the training set as the fitness function. To calculate the accuracy, we used a variant version of the *program classification map* (Zhang et al., 2003b) as the class translation rule to perform object classification. This rule situates class regions sequentially on the floating point number line. The object image will be classified to the class of the region that the program output with the object image input falls into. Class region boundaries start at some negative number, and end at the same positive number. Boundaries between the starting point and the end point are allocated with an identical interval of 1.0. For example, a five class problem would have the program classification map shown in Eq. (2) and Fig. 2.

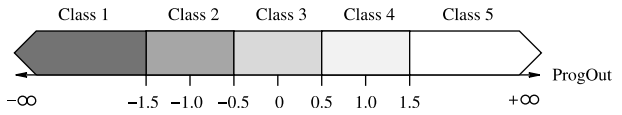

Fig. 2. An example program classification map.

$$\textbf{class} = \begin{cases} \text{Class 1}, & r < -1.5 \\ \text{Class 2}, & -1.5 \leqslant r < -0.5 \\ \text{Class 3}, & -0.5 \leqslant r < 0.5 \\ \text{Class 4}, & 0.5 \leqslant r < 1.5 \\ \text{Class 5}, & 1.5 \leqslant r \end{cases} \quad (2)$$

where r is the program output.

Although this class translation rule is easy to set and has achieved some success in several problems (Song, 2003; Zhang et al., 2003b), it has a number of disadvantages. Firstly, the ordering of classes is fixed. For binary classification problems, we only need one boundary value (usually zero) to separate the program output space into two regions, which is quite reasonable. For multiple class problems, fixing the ordering of different classes is clearly not good in many cases, since it is very difficult to set a good ordering of different classes without sufficient prior domain knowledge. Secondly, the subdivision of the real axis (program output space) is also fixed before evolution. In particular, this method separate the real axis into segments of the same size for some classes (except the "first" and the "last" classes), which often results in a long evolutionary learning time and unnecessarily complex genetic programs. While we can use different boundary values for different classes, it is also very hard to determine the appropriate sizes of different regions for hard classification problems without good expertise of a particular problem.

2.3. Parameters and termination criteria

The parameter values used in the basic GP approach are shown in Table 1. The evolutionary process is run for a fixed number (*max-generations*) of generations, unless it finds a program that solves the classification perfectly or the performance on the validation set starts falling down, at which point the evolution is terminated early.

The new approach introduced in this paper used the same program representation, program generation method, genetic operators, terminal set, function set, and the same set of parameters as the basic GP approach. However, in the new approach, we employed the Gaussian distribution and the probability theory to construct a new fitness function and to calculate the final classification accuracy in order to avoid the problems of the old fitness function in the basic GP approach. The details are presented in Sections 3 and 4.

3. Constructing fitness functions using Gaussian distribution models

In our new approach, we used Gaussian distribution and probability models to construct the fitness function of each program and used multiple programs to calculate the accuracy of a genetic program classifier.

For a set of training data, we assume that the behaviour of a program classifier is modelled using multiple Gaussian distributions, each of which corresponds to a particular class. The distribution of a class is determined by evaluating the program on the examples of the class in the training set. This was done by taking the mean and standard deviation of the program outputs for those training examples for that class.

For presentation convenience, we first use a two-class problem to describe the fitness measures, then describe the fitness function for multiclass classification problems.

3.1. Fitness measures

Fig. 3 shows three example sets of normal curves for a two class problem. If a program can successfully classify all the training examples (the ideal case), the two curves will be fairly separated (Fig. 3c); if the two curves are clearly overlapped, some examples will be incorrectly classified by the program (Fig. 3a and b). Clearly, the smaller the overlap, the better the program classifier. Two measures of overlap have been used: *area* and *distance*.

3.1.1. Area measure

In Fig. 3, the overlap area is shown in solid black. Assuming the intersection point of the two normal curves is m (m was found by a binary search), the area under the left distribution in the region $[m, \infty)$ plus the area under the right distribution in the region $(-\infty, m)$ form the overlap area.

At the beginning of evolution, the standard normal distribution $P(x)$ (Hogg and Tanis, 2000) (see Eq. (3)) is sampled from its centre (0) to some large number (i.e. 20) at regular intervals α (i.e. 0.04). The area under the

Table 1
Parameters used for GP training for the three data sets

Parameter names	Shapes	Coins	Faces	Parameter names	Shapes (%)	Coins (%)	Faces (%)
Population-size	300	500	500	Reproduction-rate	10	10	10
Initial-max-depth	3	3	3	Cross-rate	60	60	60
Max-depth	5	8	8	Mutation-rate	30	30	30
Max-generations	51	51	51	Cross-term	15	15	15
Object-size	16×16	70×70	92×112	Cross-func	85	85	85

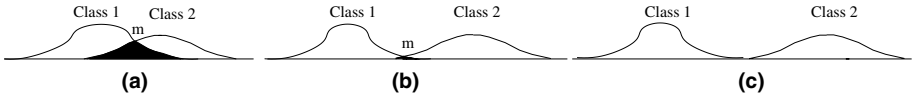

Fig. 3. Example normal distributions for a two-class problem.

distribution at each point x is approximated using Eq. (4) and the value is stored for later use to simplify the computation.

$$P(x) = \frac{\exp(\frac{-x^2}{2})}{\sqrt{2\pi}} \tag{3}$$

$$A(x) = \sum_{i=0}^{x/\alpha} \alpha P(\alpha i) \tag{4}$$

When the area under a normal distribution with mean μ and standard deviation σ is to be calculated (during evolution), Eq. (5) is used.

$$A(\mu, \sigma, x) = A\left(\frac{x-\mu}{\sigma}\right) \tag{5}$$

Accordingly, the approximation of the overlap area in Fig. 3(a) will be A_o:

$$A_o = 1 - A(\mu_1, \sigma_1, m) - A(\mu_2, \sigma_2, m) \tag{6}$$

The overlap area has a possible maximal approximation of 1.0 (where the distributions have the same mean), and a possible minimum of zero (where the distributions have different means, but both standard deviations are zero).

3.1.2. Distance measure

We also used a second measure, "weighted distribution distance" of the two distributions to evaluate the overlap, as shown in Eq. (7).

$$d = 2 \times \frac{|\mu_1 - \mu_2|}{\sigma_1 + \sigma_2} \tag{7}$$

Under this measure, the worse case is 0, when μ_1 and μ_2 are the same. In the ideal case, this distance will be very large (go to ∞).

In order to make the range of the distance measure the same as for the area measure (0 best, 1 worst), we used the following standardised distribution distance measure d_s in this approach.

$$d_s = \frac{1}{1+d} \tag{8}$$

3.2. Fitness function

For multiclass classification, there are three or more classes. The fitness function is determined by considering all the overlaps between every two classes. Assuming the number of classes is n, then there will be $C_n^2 = \frac{n\times(n-1)\times\cdots\times2\times1}{2}$ overlaps of distributions. The fitness of a program is calculated based on Eq. (9).

$$\text{fitness} = \sum_{i=1}^{C_n^2} M_i \tag{9}$$

where M_i is a fitness measure of the ith overlap of distributions of two classes, which can be either the area measure (Eq. (6)) or the distance measure (Eq. (8)).

4. Obtaining classification accuracy using multiple evolved programs

Classification accuracy is calculated by the number of objects correctly classified by a GP system as a percentage of the total number of objects in a data set. To measure which class a given pattern (object example) belongs to, we used *multiple best programs* rather than the single best program in the population (Russell and Norvig, 2003; Soule, 1999). Assuming l best programs in the population are used, the probability $Prob_c$ of a given pattern being of class c can be calculated by Eq. (10).

$$Prob_c = \prod_{i=1}^{l} P(\mu_{i,c}, \sigma_{i,c}, r_i) \tag{10}$$

where P is the normal probability density function (Hogg and Tanis, 2000) (Eq. (11)), r_i is the output result of program i with the pattern to be classified, $\mu_{i,c}$ and $\sigma_{i,c}$ are the mean and standard deviation of the outputs of program i for class c.

$$P(\mu, \sigma, x) = \frac{\exp\left(\frac{-(x-\mu)^2}{2\sigma^2}\right)}{\sigma\sqrt{2\pi}} \tag{11}$$

Based on Eq. (10), the probability of the pattern being of each class can be calculated. The class with the largest probability is used as the class of the pattern.

It is important to note that this classification criterion based on the product of the values of the probability density function for a given class for the different classifiers (programs) is only correct when the output of the different classifiers/programs are independent random variables. In many cases, however, this is not accurate. We will consider improvement of this method using a chain rule for conditional probabilities in estimating the probability densities in the future.

5. Data sets

We used three data sets providing object classification problems of increasing difficulty in the experiments. Example images are shown in Fig. 4.

The first set of images (Fig. 4a) was generated to give well defined objects against a relatively clean background. The pixels of the objects were produced using a Gaussian generator with different means and variances for each class. Three classes of 960 small objects were cut out from those

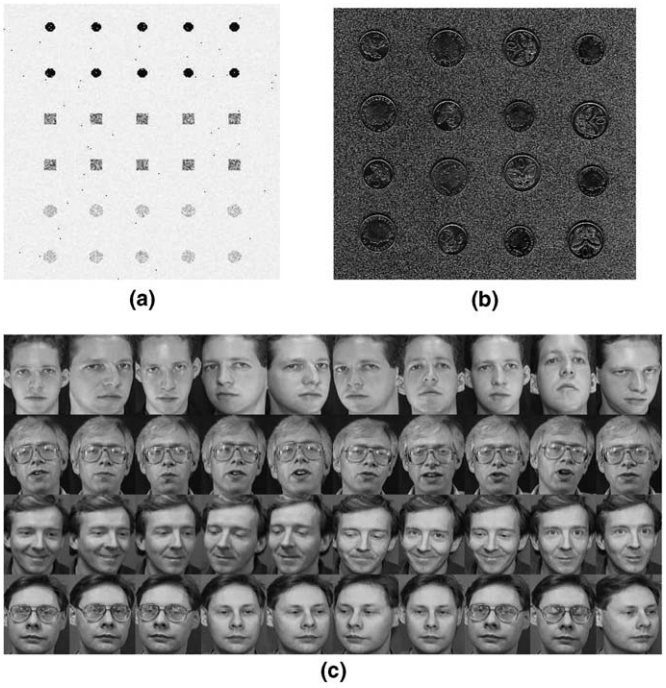

Fig. 4. Data set examples: (a) shapes, (b) coins, and (c) faces.

images to form the classification data set. The three classes are: black circles, grey squares, and light circles. For presentation convenience, this data set is referred to as *shapes*.

The second set of images (Fig. 4b) contains scanned 5 cent and 10 cent New Zealand coins. The coins were located in different places with different orientations and appeared in different sides (head and tail). In addition, the background was quite cluttered. We need to distinguish different coins with different sides from the background. Five classes of 576 object cutouts were created: 5 cent heads, 5 cent tails, 10 cent heads, 10 cent tails, and the cluttered background. Compared with the *shapes* data set, the classification problem in this data set is much harder. Although these are still regular, man-made objects, the problem is very hard due to the cluttered background and a low resolution.

The third data set consists of 40 human faces (Fig. 4c) taken at different times, varying lighting slightly, with different expressions (open/closed eyes, smiling/non-smiling) and facial details (glasses/no-glasses). These images were collected from the first four directories of the ORL face database (Samaria and Harter, 1994). All the images were taken against a dark homogeneous background with limited orientations. The task here is to distinguish those faces into the four different people.

For the shapes and the coins data sets, the objects were equally split into three separate data sets: one-third for the training set used directly for learning the genetic program classifiers, one-third for the validation set for controlling overfitting, and one-third for the test set for measuring the performance of the learned program classifiers. For the faces data set, due to the small number of images, 10-fold cross validation was applied.

6. Results and discussion

This section presents a series of results of the new method on the three object classification data sets. For all the data sets, we run 50 times with random seeds. For the shape and the coin data sets, we used 10 different training/test/validation partitions for each run (with 10 experiments) and repeat 50 times. For the face data set, we used 10-fold cross validation for each run (with 10 experiments) and repeat 50 times. Only training and test sets are used for the face data set. Accordingly, for each data set, we have 500 experiments and the average results on the test set are presented. We firstly present the overall results of the new approach compared with the basic GP approach, then describe and analyse the corresponding results against different numbers of evolved programs and different overlap measures used to perform classification.

6.1. Overall results

Table 2 shows a comparison of the average best classification results (over 500 experiments) obtained by both the new method and the basic GP approach using the same sets of features, functions and parameters. In the standard GP system, the (single) best evolved program was applied to the test set to achieve the final performance of the system. To make a fair comparison with the new approach developed in this paper, the basic approach also uses multiple "best" programs and a simple vote from these "best" programs is then applied to test set to calculate the final performance.

As can be seen from Table 2, for the shape data set, both approaches achieved very good results, reflecting the fact that the classification problem in this data set is relatively easy. However, due to the use of a less powerful class translation rule, it took the basic GP approach 11.70 generations and 3.29 s on average to find "good" program classifiers to achieve an average of 98.64% accuracy on the test set. On the other hand, the new approach used less than one generation and only 0.29 s on average to achieve almost ideal performance. This reflect the fact that the classification problem in this data set is almost trivial for the new method due to the terminals and fitness function used. Clearly, the new method outperformed the basic GP approach on this data set in terms of the convergence (number of generations), evolutionary learning time and the classification accuracy on unseen test data.

Table 2
Best results of the new and the basic GP approaches

Data set	Strategy	Generations	Time (s)	Test accuracy (%)
Shapes	Basic approach	11.70	3.29	98.64
	New approach	0.86	0.29	99.96
Coins	Basic approach	41.02	6.07	90.46
	New approach	14.26	2.27	98.60
Faces	Basic approach	8.60	0.38	86.75
	New approach	4.66	0.24	97.75

Table 3
Results for different programs and the two fitness measures

Data set	Programs used	Fitness measures					
		Generations		Time (s)		Test accuracy (%)	
		Distance	Area	Distance	Area	Distance	Area
Shapes	1	0.88	0.86	0.26	0.29	97.48	99.96
	3	0.38	0.34	0.19	0.20	99.84	99.83
	5	0.14	0.12	0.15	0.16	99.81	99.80
	10	0.04	0.04	0.14	0.15	99.79	99.78
	20	0.06	0.06	0.16	0.18	99.68	99.68
	50	0.30	0.16	0.30	0.27	99.46	99.46
Coins	1	32.98	33.78	4.55	5.37	94.23	97.49
	3	18.70	19.42	2.47	2.93	97.81	98.41
	5	14.96	15.76	1.96	2.39	98.12	98.38
	10	14.62	14.26	2.06	2.27	98.46	98.60
	20	10.82	9.06	1.60	1.57	98.40	98.25
	50	14.54	13.32	2.84	2.90	98.06	98.29
Faces	1	5.13	4.66	0.20	0.24	96.35	97.75
	3	2.09	1.73	0.08	0.10	96.95	95.90
	5	1.56	1.49	0.07	0.09	96.10	96.10
	10	1.03	0.88	0.05	0.06	95.45	94.70
	20	0.64	0.68	0.04	0.06	93.25	93.25
	50	0.25	0.18	0.03	0.04	91.20	90.30

The results on the coin and the face data sets show a similar pattern to those for the shape data set. For the coin set, the new method achieved an almost perfect average accuracy (98.60%), which is much better than that achieved by the basic GP approach (90.46%). The convergence and training processing in the new method are also much faster. For the face data set, the accuracy improvement of the new method over the basic GP approach is more than 10% ($(97.75 - 86.75)/86.75 = 12.7\%$) and evolutionary learning process for finding good classifiers is also much more efficient.

In summary, on all the three data sets, the new approach achieved very good results and always outperformed the basic approach, in terms of both classification accuracy and training time. This trend is particularly clear for relatively difficult problems such as in the coins and faces data sets. This indicates that the new approach is more effective than the basic GP approach and is more efficient to learn good classifiers for these problems.

6.2. Different programs and fitness measures

We used two fitness measures in the new approach to evaluate the programs and used multiple programs to obtain the classification results. This section is to compare the two fitness measures and to investigate how many programs should be used.

Table 3 shows a comparison of the results on the three data sets using different numbers of programs and the two fitness measures in the new approach.

As can be seen from Table 3, different numbers of programs for classification resulted in different performances. In most cases, it seems that using multiple programs led to a better accuracy and a shorter training time than using the single best evolved genetic program. At certain numbers, the method achieved a high accuracy and spent a short training time, but the numbers leading to good results for various data sets appeared to be different. It generally needs an empirical search to obtain such good numbers for different data. However, if this can improve the performance, such a search is a small price to pay.

In terms of two fitness measures, the results show that both measures achieved very good results, which were always better than the basic approach for the same problems investigated here. The area measure achieved slightly better accuracy than the distance measure in some cases, but slightly worse results in other cases. While the area measure generally required slightly fewer evaluations (generations) than the distance measure, it often took a slightly longer time to learn the good program classifiers. This is mainly because the computational complexity of the area measure is greater than the distance measure in the new algorithm.

Another interesting observation from the results is that when we only used the (single) best evolved program classifier, the area measure always achieved better classification accuracy on all the three problems investigated here. This suggests that if we only pick up the single best learned program as most GP systems do, it seems that the area measure should be employed. If we use more than five "best" programs, either measure can be chosen and applied. Further investigation on other problems needs to be carried out in the future.

7. Conclusions

The main goal of this paper was to construct the fitness function using Gaussian distributions in genetic program-

ming for multiclass object classification problems. This goal was achieved by introducing two measures for the overlaps of distributions between every two classes on the training examples. A second goal was to investigate whether this new approach was better than a basic GP approach using the same set of features (terminals) and functions. Both approaches were examined on three object classification problems of increasing difficulty. The results suggest that the new approach outperformed the basic approach in terms of both classification accuracy and training time.

Two fitness measures, *overlap area* and *distribution distance*, were developed for constructing the fitness function. Both measures achieved much better results than the basic approach on the three classification problems of increasing difficulty. The results also suggest that if only one (the best) learned program as in most GP systems, the area measure should be employed.

Unlike most existing GP approaches for classification problems, where only the best learned/evolved program classifier was applied to the object examples to measure the accuracy, this approach used multiple top programs for classification and the class with the largest probability is used as the class of the object pattern. The results suggest that the multiple program approach outperformed the single best program approach in most cases.

Compared with the basic GP approach, a major advantage of the new approach is that manually defining different boundaries for different classes over the program output space was successfully avoided. A minor disadvantage is that we need to do a bit empirical search for the best number of programs employed. Nevertheless, the results suggest that the two overlap fitness measures with different number of programs always achieved better performance than the basic GP approach.

Although developed for object classification problems, this approach is expected to be able to be applied to other classification problems.

The fitness function and the classification rule designed in this approach are based on the assumption that the output of the different classifiers/programs are independent random variables. In many cases, this is not accurate. We will consider improvement of this method using a chain rule for conditional probabilities in estimating the probability densities to investigate whether the performance can be further improved. We will also investigate the effectiveness of the new approach on other data sets, and compare this approach with other learning methods such as naive Bayes, decision trees, and neural networks in the future.

References

Andre, D., 1994. Automatically defined features: the simultaneous evolution of 2-dimensional feature detectors and an algorithm for using them. In: Kinnear, K.E. (Ed.), Advances in Genetic Programming. MIT Press, pp. 477–494.

Azimi-Sadjadi, M.R., Yao, D., Huang, Q., Dobeck, G.J., 2000. Underwater target classification using wavelet packets and neural networks. IEEE Transactions on Neural Networks 11 (3), 784–794.

Bala, J., De Jong, K., Huang, J., Vafaie, H., Wechsler, H., 1997. Using learning to facilitate the evolution of features for recognising visual concepts. Evolutionary Computation 4 (3), 297–312.

Banzhaf, W., Nordin, P., Keller, R.E., Francone, F.D., 1998. Genetic Programming: An Introduction on the Automatic Evolution of Computer Programs and its Applications. Morgan Kaufmann Publishers, Dpunkt-verlag, San Francisco, Calif, Heidelburg, ISBN 1-55860-510-X, Subject: Genetic programming (Computer science).

Caelli, T., Bischof, W.F., 1997. Machine Learning and Image Interpretation. Plenum Press, New York and London, ISBN 0-306-45761-X.

Ciesielski, V., Riley, J., 1998. An evolutionary approach to training feed forward and recurrent neural networks. in: Jain, L.C., Jain, R.K., (Eds.), Proc. Second Internat. Conf. Knowledge Based Intelligent Electronic Systems, Adelaide, April 1998, pp. 596–602.

Dunham, M.H., 2003. Data Mining: Introductory and Advanced Topics. Prentice Hall.

Eggermont, J., Eiben, A.E., van Hemert, J.I., 1999. A comparison of genetic programming variants for data classification. In: Proc. Third Symp. Intelligent Data AnalysisLecture Notes in Computer Science, 1642. Springer-Verlag.

Forsyth, David A., Ponce, Jean, 2003. Computer Vision: A Modern Approach. Prentice Hall.

Gose, E., Johnsonbaugh, R., Jost, S., 1996. Pattern Recognition and Image Analysis. Prentice Hall PTR, Upper Saddle River, NJ, ISBN 0-13-236415-8.

Gray, H.F., Maxwell, R.J., Martinez-Perez, I., Arus, C., Cerdan, S., 1996. Genetic programming for classification of brain tumours from nuclear magnetic resonance biopsy spectra. In: Koza, J.R., Goldberg, D.E., Fogel, D.B., Riolo, R.L. (Eds.), Genetic Programming 1996: Proc. First Annual Conf., 28–31 July, 1996. MIT Press, Stanford University, CA, USA, p. 424.

Hogg, R.V., Tanis, E.A., 2000. Probability and Statistical Inference, sixth ed. Prentice Hall.

Howard, A., Padgett, C., Liebe, C.C., 1998. A multi-stage neural network for automatic target detection. In: 1998 IEEE World Congress on Computational Intelligence—IJCNN'98, Anchorage, Alaska, 1998, pp. 231–236. ISBN: 0-7803-4859-1/98.

Howard, D., Roberts, S.C., Brankin, R., 1999. Target detection in SAR imagery by genetic programming. Advances in Engineering Software 30, 303–311.

Howard, D., Roberts, S.C., Ryan, C., 2002. The boru data crawler for object detection tasks in machine vision. In: Cagnoni, S., Gottlieb, J., Hart, E., Middendorf, M., Raidl, G. (Eds.), Applications of Evolutionary Computing, Proc. EvoWorkshops2002: EvoCOP, EvoIASP, EvoSTim, 3–4 April 2002, Lecture Notes in Computer Science, vol. 2279. Springer-Verlag, Kinsale, Ireland, pp. 220–230.

Huang, J.-S., Liu, H.-C., 1997. Object recognition using genetic algorithms with a Hopfield's neural model. Expert Systems with Applications 13 (3), 191–199.

Korning, P.G., 1995. Training neural networks by means of genetic algorithms working on very long chromosomes. International Journal of Neural Systems 6 (3), 299–316.

Koza, J.R., 1992. Genetic Programming: on the Programming of Computers by Means of Natural Selection. MIT Press, London, England, Cambridge, Mass.

Koza, J.R., 1994. Genetic Programming II: Automatic Discovery of Reusable Programs. MIT Press, London, England, Cambridge, Mass.

LeCun, Y., Jackel, L.D., Boser, B., Denker, J.S., Graf, H.P., Guyon, I., Henderson, D., Howard, R.E., Hibbard, W., 1989. Handwritten digit recognition: application of neural network chips and automatic learning. IEEE Communications Magazine (November), 41–46.

LeCun, Y., Bottou, L., Bengio, Y., Haffner, P., 2001. Gradient-based learning applied to document recognition. In: Haykin, S., Kosko, B. (Eds.), Intelligent Signal Processing. IEEE Press, pp. 306–351.

Loveard, T., Ciesielski, V., 2001. Representing classification problems in genetic programmingProc. Congress on Evolutionary Computation, COEX, World Trade Center, 159 Samseong-dong, Gangnam-gu, Seoul, Korea, 27–30 May 2001, vol. 2. IEEE Press, pp. 1070–1077.

de Ridder, D., Hoekstra, A., Duin, R.P.W., 1996. Feature extraction in shared weights neural networks. In: Proc. Second Annual Conf. Advanced School for Computing and imaging, ASCI, Delft, June 1996, pp. 289–294.

Russell, S., Norvig, P., 2003. Artificial Intelligence, A modern Approach, second ed. Prentice Hall.

Samaria, F., Harter, A., 1994. Parameterisation of a stochastic model for human face identification. In: 2nd IEEE Workshop on Applications of Computer Vision, Sarasota (Florida), July 1994. ORL database is available from: <www.cam-orl.co.uk/facedatabase.html>.

Sherrah, J.R., Bogner, R.E., Bouzerdoum, A., 1997. The evolutionary pre-processor: automatic feature extraction for supervised classification using genetic programming. In: Koza, J.R., Deb, K., Dorigo, M., Fogel, D.B., Garzon, M., Iba, H., Riolo, R.L. (Eds.), Genetic Programming 1997: Proc. Second Annual Conf., 13–16 July. Morgan Kaufmann, Stanford University, CA, USA, pp. 304–312.

Smart, W., Zhang, M., 2003. Classification strategies for image classification in genetic programming. In: Bailey, D. (Ed.), Proceeding of Image and Vision Computing Conference. Palmerston North, New Zealand, pp. 402–407.

Song, A., Loveard, T., Ciesielski, V., 2001. Towards genetic programming for texture classification. In: Proc. 14th Australian Joint Conf. Artificial Intelligence. Springer Verlag, pp. 461–472.

Song, A., Ciesielski, V., Williams, H., 2002. Texture classifiers generated by genetic programming. In: Fogel, D.B., El-Sharkawi, M.A., Yao, X., Greenwood, G., Iba, H., Marrow, P., Shackleton, M. (Eds.), Proc. 20002 Congress on Evolutionary Computation CEC2002. IEEE Press, pp. 243–248.

Song, A., 2003. Texture Classification: A Genetic Programming Approach. Ph.D. Thesis, Department of Computer Science, RMIT University, Melbourne, Australia, 2003.

Soule, T., 1999. Voting teams: a cooperative approach to non-typical problems using genetic programming. In: Banzhaf, W., Daida, J., Eiben, A.E., Garzon, M.H., Honavar, V., Jakiela, M., Smith, R.E. (Eds.), Proc. Genetic and Evolutionary Computation Conf., 13–17 July 1999, vol. 1. Morgan Kaufmann, Orlando, Florida, USA, pp. 916–922.

Stahl, C., Aerospace, Daimler Chrysler, Schoppmann, P., 2000. Advanced automatic target recognition for police helicopter missions. In: Sadjadi, F.A. (Ed.), Proc. SPIE, Automatic Target Recognition X, vol. 4050, pp. 61–68.

Tackett, W.A., 1993. Genetic programming for feature discovery and image discrimination. In: Forrest, S. (Ed.), Proc. 5th Internat. Conf. Genetic Algorithms, ICGA-93, 17–21 July, 1993. University of Illinois at Urbana-Champaign, Morgan Kaufmann, pp. 303–309.

Teller, A., Veloso, M., 1995. A controlled experiment: evolution for learning difficult image classification. In: Carlos, P.-F., Mamede, N.J. (Eds.), LNAI, 3–6 October 1995, vol. 990. Springer Verlag, Berlin, pp. 165–176.

Valentin, D., Abdi, H., O'Toole, A., 1994. Categorization and identification of human face images by neural networks: a review of linear auto-associator and principal component approaches. Journal of Biological Systems 2 (3), 413–429.

Verma, B., 1998. A neural network based technique to locate and classify microcalcifications in digital mammograms. 1998 IEEE World Congress on Computational Intelligence—IJCNN'98. Anchorage, Alaska. IEEE, ISBN: 0-7803-4859-1/98.

Wessels, T., Omlin, C.W., 2000. A hybrid system for signature verification. In: Proc. IEEE-INNS-ENNS Internat. Joint Conf. Neural Networks (IJCNN'00), vol. V, Como, Italy, July 2000.

Winkeler, J.F., Manjunath, B.S., 1997. Genetic programming for object detection. In: Koza, J.R., Deb, K., Dorigo, M., Fogel, D.B., Garzon, M., Iba, H., Riolo, R.L. (Eds.), Genetic Programming 1997: Proc. Second Annual Conf., 13–16 July. Stanford University, Morgan Kaufmann, CA, USA, pp. 330–335.

Winter, P., Yang, W., Sokhansanj, S., Wood, H., 1996. Discrimination of hard-to-pop popcorn kernels by machine vision and neural network. In: ASAE/CSAE meeting, Saskatoon, Canada, September 1996. Paper No. MANSASK 96-107.

Wong, Y.C., Sundareshan, M.K., 1998. Data fusion and tracking of complex target maneuvers with a simplex-trained neural network-based architecture. In: 1998 IEEE World Congress on Computational Intelligence—IJCNN'98, Anchorage, Alaska, May 1998, pp. 1024–1029. ISBN: 0-7803-4859-1/98.

Yao, X., Liu, Y., 1997. A new evolutionary system for evolving artificial neural networks. IEEE Transactions on Neural Networks 8 (3), 694–713.

Zhang, M., Ciesielski, V., 1999. Genetic programming for multiple class object detection. In: Foo, N. (Ed.), Proc. 12th Australian Joint Conf. Artificial Intelligence (AI'99), Sydney, Australia, December 1999, Lecture Notes in Artificial Intelligence (LNAI), vol. 1747. Springer-Verlag, Berlin, Heidelberg, pp. 180–192.

Zhang, M., Andreae, P., Pritchard, M., 2003a. Pixel statistics and false alarm area in genetic programming for object detection. In: Cagnoni, S. (Ed.), Applications of Evolutionary Computing, Lecture Notes in Computer Science, LNCS, 2611. Springer-Verlag, pp. 455–466.

Zhang, M., Ciesielski, V., Andreae, P., 2003b. A domain independent window-approach to multiclass object detection using genetic programming. EURASIP Journal on Signal Processing. Special Issue on Genetic and Evolutionary Computation for Signal Processing and Image Analysis 2003 (8), 841–859.

Zhang, M., Smart, W., 2004. Multiclass object classification using genetic programming. In: Raidl, G.R., Cagnoni, S., Branke, J., Corne, D.W., Drechsler, R., Jin, Y., Johnson, C., Machado, P., Marchiori, E., Rothlauf, F., Smith, G.D., Squillero, G. (Eds.), Applications of Evolutionary Computing, EvoWorkshops2004: EvoBIO, EvoCOMNET, EvoHOT, EvoIASP, EvoMUSART, EvoSTOC, 5–7 April, Lecture Notes in Computer Science, vol. 3005. Springer Verlag, Coimbra, Portugal, pp. 367–376.